Guion de trabajo tutorizado

Bases de datos masivamente paralelas – Casos prácticos

Gestión y visualización de datos masivos

Profesor Dr. Carlos D. Barranco González
Máster Big Data & Business Analytics (MBDA)





Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Guion oficial: «MBDA - Bases de datos masivamente paralelas - Casos Prácticos - Actividad.pdf» (Campus Virtual UPO).

Objetivos

• Conocer los procedimientos para el despliegue de un cluster que ofrezca los servicios de un motor de datos masivamente paralelo, empleando HBase como ejemplo.

• Conocer el funcionamiento de HBase ante recuperación de desastres.

• Practicar con el sistema de particionado y asignación de nodos de HBase.

Actividades

A1. (90 minutos – 2.5 puntos) Hasta el momento, hemos usado todos los motores de bases de datos de la asignatura, y en particular HBase, en un solo nodo. Como se vio en las sesiones de clase, el principal motivo por el que tienen tanto auge los motores de datos catalogados dentro de la categoría de NoSQL es su facilidad para la creación de clusters de nodos que den en conjunto un servicio de motor de datos distribuido, de tal forma que la carga se reparta entre ellos y, a la vez, nos proporcionen una alta disponibilidad.

En este ejercicio simularemos en una sola máquina, gracias al modo de ejecución pseudodistributed de HBase, la creación de tal cluster.

Esta actividad tiene dos opciones en función del perfil que se ajuste más a sus intereses. Tenga en cuenta que es libre de hacer las dos opciones, aunque sólo se exige una, pero en ese caso el tiempo estimado será el doble.

Opción 1 (Perfil analista):

El proceso de instalación de HBase es un ejercicio técnico que requiere alguna soltura con sistemas Linux. Es muy recomendable hacerlo, pero, quizás, las cuestiones técnicas asociadas podrían prevenir que un alumno, sin tanto interés o pericia en éstas, no llegue al punto en el que experimente con un sistema HBase operativo y pueda realizar el resto de actividades. Es por ello que se plantea la siguiente opción.

Para realizar esta actividad, deberá seguir los pasos planteados en [5]. Esto le permitirá tener una instalación de HBase pseudo-distribuido, y poder jugar un poco con el servidor. Tenga en cuenta que en [5] se va a más allá del objeto de esta actividad, llegando los contenidos a alinearse con los de las actividades 2, 3 y 4, como referencia para los alumnos.

El resultado de esta actividad, que ha de reflejar en la entrega, consistirá en ilustrar el proceso que ha seguido para la puesta en marcha de un HBase pseudo-distribuido, y las evidencias necesarias para demostrar que el citado motor está operativo. Incluya para ello tanto capturas de la consola (enseñando, p. ej. la salida del comando status, donde se pueden ver los servidores de región que hay activos y de maestros de respaldo) como del interfaz web de HBase (especialmente en la parte donde se indica el listado de servidores de región y de maestros), así como la salida del comando jps. Como siempre, toda captura irá con su explicación textual correspondiente.

Opción 2 (Perfil desarrollador):

Para realizar una instalación pseudo-distribuida, seguiremos las instrucciones indicadas en [1] para la creación de una instalación de este tipo y partiremos de la instalación y la máquina virtual que desarrollamos en el trabajo tutorizado dedicado a las bases de datos orientadas a columnas. Puede

tener una referencia de cómo ha de funcionar el sistema siguiendo la anterior actividad para el perfil analista, y particularmente con el video en [5].

Para realizar esta actividad, tenga en cuenta que aunque en las instrucciones se indica que se puede emplear el sistema de ficheros HDFS para el almacenamiento de los datos en modo pseudo-distribuido, esto es opcional y también se permite el uso del sistema de ficheros local. Por simplicidad, optaremos por esta vía, ya que se va a hacer uso del modo de ejecución pseudo-distribuido, por lo que puede ignorar los comentarios de las instrucciones a este respecto. No obstante, y a modo opcional, si el alumno desea realizar la instalación sobre HDFS es libre de hacerlo y será valorado positivamente. Tenga en cuenta que es interesante contar con HDFS porque permite una simulación más completa de lo que es un entorno real, en el que siempre será necesario contar con HDFS para distribuir los datos entre los nodos, pero el tiempo dedicado a esta actividad será superior y la carga de la máquina también será más elevada.

Por el momento, y para este ejercicio, nos conformaremos con tener HBase funcionando en modo pseudo-distributed, pero no añadiremos más nodos maestros ni más servidores de región a los que se crean por defecto (uno de cada).

Como resultado del ejercicio el alumno deberá de realizar una documentación en el que se muestren evidencias de los pasos realizados en el proceso de configuración y del funcionamiento de tal instalación, contando entre ellas, con capturas de pantalla, descripción del contenido de las mismas y, en particular, la información que se obtiene mediante el comando jps, la información que el interfaz web del servidor nos brinda, así como la salida del comando status de la consola de HBase.

Tenga en cuenta que para realizar este tipo de instalación, según lo indicado en [6], se debe dotar a la máquina virtual de servicio SSH (puede ver cómo instalar un servicio SSH en [2]). Además, no olvide la configuración de JAVA_HOME en hbase-env.sh. En todo caso, puede hacer uso de los ficheros de log y salida que ofrece HBase en el directorio log para detectar cualquier tipo de problema.

A2. (30 minutos – 0.7 puntos) Esta es una actividad sencilla en la que veremos cómo se comporta HBase cuando muere su único nodo maestro, simulando así una caída de nodos críticos de un cluster. Para ello, partiendo de un sistema HBase pseudo-distribuido, con un maestro y un servidor de regiones (misma configuración que la obtenida en la Actividad 1), localizaremos con jps el identificador del proceso de maestro de HBase y lo mataremos con el comando kill (puede ver un ejemplo similar de localización de procesos en [5]), todo ello mientras estamos conectados con la consola.

El resultado de esta actividad consistirá en recoger evidencias antes y después del proceso descrito, especialmente la salida de jps, el resultado del comando status de la consola, así como de la interfaz web. ¿Qué pasará cuando el maestro muera y se manden comandos como ‘status’ o ‘create’ para crear una tabla? El alumno recogerá esas evidencias en un documento.

A3. (60 minutos – 1.7 puntos) La instalación, tal y como la tenemos ahora, sólo levanta un maestro y un servidor de región (si viene de la actividad anterior, tendrá que reiniciar HBase para que vuelva a levantar el maestro por defecto). En este ejercicio levantaremos más maestros (con un par será suficiente), que pasarán a ser maestros de respaldo, tal y como se sugiere en [5]. Para ello, puede ejecutar un comando similar al siguiente:

```
$ local-master-backup.sh start 2 3 4
```

El alumno realizará el proceso de crear nuevos maestros y recogerá evidencias del resultado, tanto de la salida del comando jps, como la información sobre maestros de respaldo de la interfaz web y la que proporciona el comando status de la consola.

Tenga en cuenta que si tiene problemas a la hora de levantar nuevos maestros, esto puede deberse a que debe revisar la posibilidad de conexión con SSH a la máquina, si está usando una instalación realizada por usted mismo. En este caso, puede emplear la imagen de Docker de la Actividad 1, o realizar la configuración necesaria en su instalación.

En caso de éxito, el alumno matará al maestro principal y observará el comportamiento del sistema a este respecto, proporcionando evidencias del cambio en los procesos (jps) y en la interfaz web, así como mediante la salida del comando status donde quede patente ese cambio de maestro. Tenga en cuenta que cuando el maestro principal muere, el puerto de la interfaz web que debe usar para acceder es el del maestro de respaldo que le sucede (esta información la puede obtener de la interfaz web previamente a la muerte del maestro principal).

En algunos casos minoritarios, dependiendo de la instalación que haya realizado (si la hizo usted mismo) o de la capacidad de memoria disponible en su máquina, es posible que los maestros de respaldo se levanten y a los pocos segundos mueran. No se preocupe por eso, pero en este caso deberá justificar por qué ocurre esto examinando los mensajes en el log de cada uno de los maestros de respaldo.

El alumno recopilará dichas evidencias en un documento describiendo el contenido de las mismas.

A4. (60 minutos – 1.7 puntos) Ha llegado el momento, en esta actividad, de incluir más servidores de región en nuestro cluster simulado (Por ejemplo, 4 más, para que tengamos 5). Para ello, seguiremos un procedimiento similar al que se puede ver en [5].

De la misma forma que el resto de ejercicios, el alumno recogerá evidencias del proceso (fundamentalmente con los resultados de jps, el comando status de la consola y el interfaz web), antes de realizar el proceso de inicio de los nuevos servidores de región y después, y reunirá las mismas en un documento.

A5. (60 minutos – 1.7 puntos) Ya tenemos un cluster a nuestra disposición y ha llegado el momento de que lo alimentemos con datos. Obviamente en esta ocasión no sería práctico alimentarlo con millones de datos (aunque si desarrolló el generador de datos aleatorios en Ruby en el anterior guion, puede usarlo), pero comenzaremos con 10. De esta forma, el alumno se conectará al cluster, creará una tabla de ejemplo y la alimentará con 10 entradas. Tenga especial cuidado con la selección de la clave, ya que nos interesa que éstas sean lejanas entre sí, lexicográficamente hablando, para facilitar la siguiente actividad.

Hecha esa creación de registros ¿Podríamos ver en cuántas regiones ha sido dividida la tabla? ¿Por qué se divide en ese número de regiones? ¿Qué evidencias da la interfaz web al respecto (sobre las regiones en las que está divida una tabla)? ¿Cuál es el cometido de la tabla META (hbase:meta) y qué significa el mismo en relación a esta cuestión? Haga un scan de la tabla META, vuelque su contenido en una captura e interprete la totalidad del mismo ¿Qué fuentes ha usado para interpretar el contenido de la tabla META? (busque alguna fuente en Internet para interpretar el contenido de META y referénciela en su respuesta).

El alumno realizará la actividad, dejando evidencias de los resultados, y contestará a las preguntas planteadas una por una y de forma separada y específica.

A6. (60 minutos – 1.7 puntos) Como hemos comentado, la cantidad de datos que por el momento gestionará HBase no desvelará todo su potencial y, de hecho, hasta ahora no hemos visto funcionando el sistema de particionado. En esta actividad probaremos tal funcionamiento.

Sabemos que HBase tiene un sistema de particionado automático, pero éste se puede forzar. Para ello, usaremos el comando split de la consola (emplee la referencia [3] para consultar su funcionamiento).

Concretamente, nos proponemos dividir la región en la que se almacena la tabla creada en la A5 en dos partes. Por tanto, observe el funcionamiento del comando y elija una clave de partición adecuada en función de los identificadores que empleó para sus datos. No es necesario que la división sea exactamente por la mitad, pero es recomendable que así sea por simplicidad. Recoja evidencias de que dicha división se ha realizado (recuerde recoger las evidencias antes y después del proceso), empleando tanto los datos de la tabla META (que deberá interpretar por escrito, además de mostrarlos) como los ofrecidos por el interfaz web y determine en qué servidores de regiones están siendo almacenadas cada una de las regiones.

Conseguido lo anterior, moveremos una región de un servidor de regiones a otro, simulando el caso en que, por rendimiento, el cluster, o nosotros como administradores, toma esta decisión para mejorar el mismo. Para ello, usaremos el comando move con objeto de forzar el movimiento de una región de un servidor de región a otro. Tenga en cuenta las instrucciones dadas en [3] para este comando. Podrá conseguir el identificador de la región empleando el interfaz web o META, y el identificador del servidor región de destino empleando dicho interfaz web. Reúna evidencias del proceso y los resultados.

Además de mover forzosamente una región con el comando move, y tras esto, simularemos la caída de un nodo (que sea servidor de regiones y que tenga alguna región almacenada) de nuestro cluster, y veremos a dónde van a parar las regiones almacenadas en él. Para ello, identifique un servidor de regiones y las regiones que almacena y mátelo (como en ejercicios anteriores). En el caso que desee matar un servidor de regiones que no almacene ninguna región, puede mover (antes de matarlo) una de las existentes al servidor de región elegido. Recuerde que puede localizar el pid (process identifier) de un servidor de región concreto según las instrucciones indicadas en [5]. Obtenga evidencias antes y después de matar al servidor de regiones, debiendo quedar claramente ilustrada la antigua y nueva localización de las regiones afectadas. Tenga en consideración que el movimiento de la región podría tardar algunos minutos. Además, acceda a los datos de la tabla almacenada en la región afectada (con un get o scan) para demostrar que éstos no se han perdido.

Finalmente, buscaremos unir de nuevo las regiones que resultaron de la división en una sola. Esta operación se puede conseguir sirviéndonos del comando merge_region, siguiendo un procedimiento similar al descrito en [4]. Realice dicha operación y reúna evidencias del resultado ¿En cuantas regiones quedó la tabla? ¿En qué servidor está localizada?

Tenga en cuenta que el resultado de esta actividad es la reunión de todas las evidencias que demuestren el antes y el después de las operaciones, pero recuerde que todas ellas se deben comentar textualmente, a fin de indicar qué demuestran.

A7. (Opcional – 180 minutos – 3.5 puntos extra) Esta actividad es muy técnica, por lo que supondrá un reto muy interesante. No obstante, requiere disponer de unas habilidades técnicas adecuadas, y, por tanto, se establece como opcional.

En el actual guion, ha llegado a ejecutar HBase en un modo pseudo-distribuido, debido a que estamos trabajando con una sola máquina. No obstante, si no dispone de varios modos para ejecutar HBase en modo completamente distribuido, puede simular los mismos ejecutando distintas máquinas virtuales, contenedores docker o nodos en AWS (u otro proveedor cloud), siendo cada una de ellas un nodo. En esta actividad se propone que, siguiendo este planteamiento para simular nodos y las instrucciones en [1] para un entorno completamente distribuido, consiga una instalación completamente distribuida. Tenga en cuenta que para ello, será necesario que los nodos tengan también una instalación de HDFS funcional.

Modo de entrega

- Los alumnos entregarán un documento PDF a través de una actividad programada en el Aula Virtual empleando la plantilla suministrada.

- Los alumnos tienen hasta el segundo lunes, desde la impartición de la clase, para devolver el cuestionario.

Bibliografía

1. Apache HBase ™ Reference Guide - Getting Started https://hbase.apache.org/book.html#_getting_started

2. SSH Setup For HBase http://techvineyard.blogspot.com.es/2011/01/ssh-setup-for-hbase.html

3. HBase shell commands https://learnhbase.wordpress.com/2013/03/02/hbase-shell-commands/

4. HBase regions merge. Le Cluster de Laurent, 2015. Original: http://lecluster.delaurent.com/hbase-regions-merge/ Ahora disponible en: https://web.archive.org/web/20190905105358/http://lecluster.delaurent.com/hbaseregions-merge/

5. HBase pseudo-distribuido. Carlos D. Barranco. https://youtu.be/kbRDh5WwG_A

6. Apache HBase Reference Guide - 4. Basic Prerequisites https://hbase.apache.org/book.html#basic.prerequisites

RESOLUCIÓN

Entorno utilizado

Se ha trabajado sobre una máquina Linux (Ubuntu 24.04, 2 vCPU, 7 GB de RAM) con OpenJDK 17 y **Apache HBase 2.6.6** (binario oficial `hbase-2.6.6-bin.tar.gz` de archive.apache.org) desempaquetado en `/opt/hbase`. Se sigue la **Opción 2 (perfil desarrollador)** de la A1: instalación propia en modo *pseudo-distribuido* sobre el sistema de ficheros local (sin HDFS), con el ZooKeeper gestionado por el propio HBase.

A1. Instalación de HBase en modo pseudo-distribuido

1) Configuración. El modo pseudo-distribuido se activa con `hbase.cluster.distributed=true`: cada demonio (ZooKeeper, HMaster y HRegionServer) corre en su propia JVM, igual que en un clúster real, pero todos en la misma máquina. Como se indica en el enunciado, se usa el sistema de ficheros local (`hbase.rootdir=file:///...`) y se desactiva la comprobación `hbase.unsafe.stream.capability.enforce`, necesaria cuando no hay HDFS. En `hbase-env.sh` se fija `JAVA_HOME`, se indica que HBase gestione su ZooKeeper (`HBASE_MANAGES_ZK=true`) y se limita el heap a 512 MB. Los scripts de arranque usan SSH hacia `localhost`, por lo que se instala `openssh-server` y se configura el acceso por clave sin contraseña (última orden de la figura).

```
aramber@localhost:~$ java -version
openjdk version "21.0.10" 2026-01-20
OpenJDK Runtime Environment (build 21.0.10+7-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.10+7-Ubuntu-124.04, mixed mode, sharing)
aramber@localhost:~$ cat /opt/hbase/conf/hbase-site.xml
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
  <property><name>hbase.cluster.distributed</name><value>true</value></property>
  <property><name>hbase.rootdir</name><value>file:///opt/hbase-data/hbase</value></property>
  <property><name>hbase.zookeeper.property.dataDir</name><value>/opt/hbase-data/zookeeper</value></property>
  <property><name>hbase.tmp.dir</name><value>/opt/hbase-data/tmp</value></property>
  <property><name>hbase.unsafe.stream.capability.enforce</name><value>false</value></property>
  <property><name>hbase.zookeeper.quorum</name><value>localhost</value></property>
</configuration>
aramber@localhost:~$ grep -E '^export' /opt/hbase/conf/hbase-env.sh
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export HBASE_MANAGES_ZK=true
export HBASE_HEAPSIZE=512m
export JAVA_TOOL_OPTIONS=
aramber@localhost:~$ cat /opt/hbase/conf/regionservers
localhost
aramber@localhost:~$ ssh localhost 'hostname && echo SSH sin contraseña OK'
vm
SSH sin contraseña OK
```

Figura 1. Versión de Java, `hbase-site.xml`, variables de `hbase-env.sh`, fichero `regionservers` y comprobación de SSH sin contraseña a localhost.

2) Arranque. `start-hbase.sh` levanta, por este orden, el ZooKeeper (`HQuorumPeer`), el maestro (`HMaster`) y el servidor de regiones (`HRegionServer`).

```
aramber@localhost:~$ /opt/hbase/bin/start-hbase.sh
localhost: running zookeeper, logging to /opt/hbase/bin/../logs/hbase-root-zookeeper-vm.out
running master, logging to /opt/hbase/bin/../logs/hbase--master-vm.out
: running regionserver, logging to /opt/hbase/bin/../logs/hbase--regionserver-vm.out
```

Figura 2. Arranque del clúster pseudo-distribuido con `start-hbase.sh`.

3) Evidencias de funcionamiento. La salida de `jps` muestra los tres procesos Java del clúster, cada uno en su propia JVM (lo que distingue el modo pseudo-distribuido del *standalone*, donde todo corre en un único proceso `HMaster`):

```
aramber@localhost:~$ jps -l | sort -k2
2889 jdk.jcmd/sun.tools.jps.Jps
2292 org.apache.hadoop.hbase.master.HMaster
2431 org.apache.hadoop.hbase.regionserver.HRegionServer
2177 org.apache.hadoop.hbase.zookeeper.HQuorumPeer
```

Figura 3. Salida de `jps -l`: HQuorumPeer (ZooKeeper), HMaster y HRegionServer.

En la consola de HBase, `status` informa de **1 maestro activo, 0 maestros de respaldo, 1 servidor de regiones vivo y 0 caídos**. `status 'simple'` detalla el maestro activo (`localhost:16000`) y el servidor de regiones (`localhost:16020`) con sus métricas: aloja 2 regiones, que son las de las tablas de sistema `hbase:meta` y `hbase:namespace`.

```
aramber@localhost:~$ echo -e "status\nstatus 'simple'\nversion" | hbase shell -n 2>/dev/null
hbase:001:0> status
1 active master, 0 backup masters, 1 servers, 0 dead, 2.0000 average load
Took 0.5083 seconds
hbase:002:0> status 'simple'
active master:  localhost:16000 1789811153863
0 backup masters
1 live servers
    localhost:16020 1789811155600
        requestsPerSecond=0.0, numberOfOnlineRegions=2, usedHeapMB=48, maxHeapMB=512, numberOfStores=4, numberOfStoref
iles=0, storefileUncompressedSizeMB=0, storefileSizeMB=0, memstoreSizeMB=2, storefileIndexSizeKB=0, readRequestsCount=
10, filteredReadRequestsCount=0, writeRequestsCount=7, rootIndexSizeKB=0, totalStaticIndexSizeKB=0, totalStaticBloomSi
zeKB=0, totalCompactingKVs=0, currentCompactedKVs=0, compactionProgressPct=NaN, coprocessors=[MultiRowMutationEndpoint
]
0 dead servers
Aggregate load: 0, regions: 2
Took 0.0214 seconds
hbase:003:0> version
2.6.6, rd5f63f523c065f963992ea4e74db268d17816631, Thu Jun  4 11:07:52 PDT 2026
Took 0.0012 seconds
hbase:004:0>
```

Figura 4. Comandos `status`, `status 'simple'` y `version` en la consola de HBase.

La interfaz web del maestro (puerto 16010) confirma lo mismo: un servidor de regiones en estado *Normal* con 2 regiones y ningún maestro de respaldo.

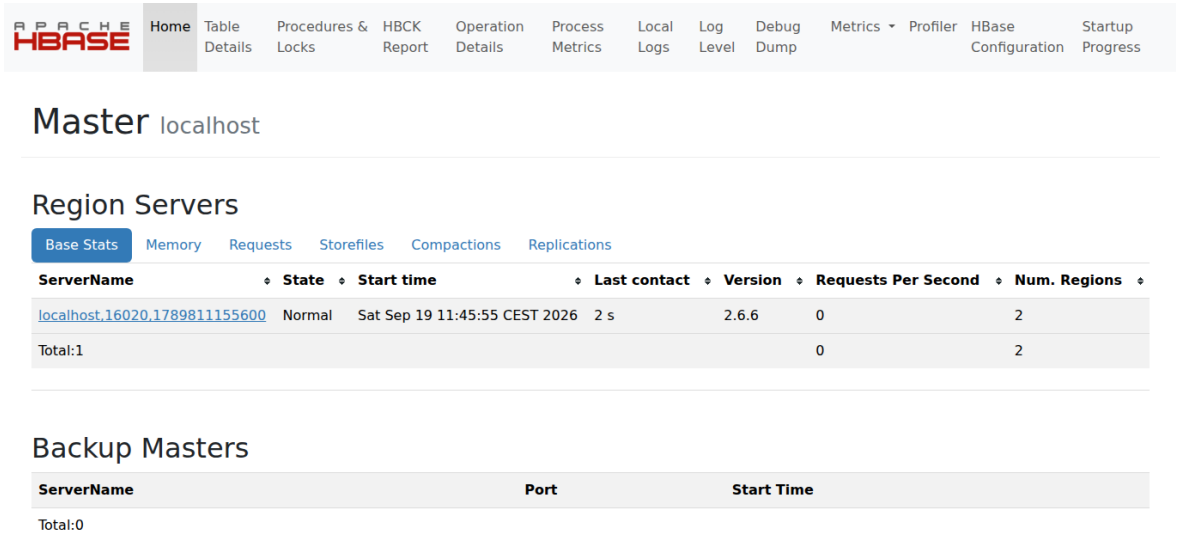


Figura 5. Interfaz web del HMaster (http://localhost:16010): listado de Region Servers y Backup Masters.

A2. Caída del único maestro

Antes: el clúster tiene un maestro (PID 2292), un servidor de regiones y el ZooKeeper; `status` es correcto.

```
aramber@localhost:~$ jps | grep -v Jps
2177 HQuorumPeer
2292 HMaster
2431 HRegionServer
aramber@localhost:~$ echo status | hbase shell -n 2>/dev/null | grep -v '^Took'
hbase:001:0> status
1 active master, 0 backup masters, 1 servers, 0 dead, 2.0000 average load
hbase:002:0>
```

Figura 6. Estado previo: `jps` y `status` con el maestro activo.

Se localiza el PID del `HMaster` con `jps` y se mata con `kill -9`. Tras ello, `jps` sólo muestra ZooKeeper y el servidor de regiones.

```
aramber@localhost:~$ kill -9 2292    # PID del HMaster obtenido con jps
2177 HQuorumPeer
2431 HRegionServer
```

Figura 7. Muerte forzada del proceso HMaster.

Después: tanto `status` como `create` fallan con `KeeperErrorCode = NoNode for /hbase/master`. El cliente pregunta a ZooKeeper quién es el maestro activo (nodo efímero `/hbase/master`); al morir el proceso, su sesión con ZooKeeper expira y el nodo desaparece. Las operaciones de **administración** (`status`, `create`, `alter`, `drop`, `balance`…) necesitan al maestro, así que no pueden ejecutarse. Las lecturas y escrituras de datos, en cambio, las atienden directamente los servidores de regiones, que siguen vivos. Aun así, el clúster ha perdido su capacidad de gestión y no tolera otro fallo, porque nadie podría reasignar las regiones de un servidor caído: es un punto único de fallo.

```
aramber@localhost:~$ echo status | timeout 90 hbase shell -n 2>&1 | grep -v 'at org\|at java\|at $' | head -12
hbase:001:0> status
Took 0.0806 secondsUnhandled Java exception: org.apache.zookeeper.KeeperException$NoNodeException: KeeperErrorCode = N
oNode for /hbase/master
org.apache.zookeeper.KeeperException$NoNodeException: KeeperErrorCode = NoNode for /hbase/master
```

Figura 8. `status` sin maestro activo.

```
aramber@localhost:~$ echo "create 'prueba_sin_master', 'cf'" | timeout 120 hbase shell -n 2>&1 | grep -vE '^\s+(at|[a-zA-Z_]+ at) ' | hea
d -20; echo "(código de salida: $?)"
hbase:001:0> create 'prueba_sin_master', 'cf'
Took 0.0657 secondsUnhandled Java exception: org.apache.zookeeper.KeeperException$NoNodeException: KeeperErrorCode = N
oNode for /hbase/master
org.apache.zookeeper.KeeperException$NoNodeException: KeeperErrorCode = NoNode for /hbase/master

(código de salida: 0)
```

Figura 9. `create` sin maestro activo: la operación de administración falla.

La interfaz web (servida por el propio maestro) deja de responder:

```
aramber@localhost:~$ curl -sS http://localhost:16010/master-status
curl: (7) Failed to connect to localhost port 16010 after 0 ms: Couldn't connect to server
```

Figura 10. La interfaz web en el puerto 16010 ya no está disponible.

Para continuar, el maestro se relanza con `hbase-daemon.sh start master` y el clúster vuelve a su estado normal:

```
aramber@localhost:~$ hbase-daemon.sh start master
running master, logging to /opt/hbase/bin/../logs/hbase-root-master-vm.out
aramber@localhost:~$ sleep 20; jps | grep -v Jps
2177 HQuorumPeer
4347 HMaster
2431 HRegionServer
aramber@localhost:~$ echo status | hbase shell -n 2>/dev/null | grep -v Took
hbase:001:0> status
1 active master, 0 backup masters, 1 servers, 0 dead, 2.0000 average load
hbase:002:0>
```

Figura 11. Recuperación del maestro.

A3. Maestros de respaldo y conmutación por error

Con `local-master-backup.sh start 2 3` se levantan dos maestros adicionales con un *offset* de puertos (+2 y +3): RPC en 16002/16003 y web en 16012/16013. Arrancan con `--backup`: se registran en ZooKeeper bajo `/hbase/backup-masters` y esperan. `status 'simple'` lo confirma: **1 maestro activo (16000) y 2 de respaldo (16002, 16003)**.

```
aramber@localhost:~$ local-master-backup.sh start 2 3
running master, logging to /opt/hbase/bin/../logs/hbase-root-2-master-vm.out
running master, logging to /opt/hbase/bin/../logs/hbase-root-3-master-vm.out
aramber@localhost:~$ sleep 25; jps -m | grep -v Jps
2177 HQuorumPeer start
4963 HMaster -D hbase.master.port=16002 -D hbase.master.info.port=16012 -D hbase.regionserver.port=16022 -D hbase.regi
onserver.info.port=16032 --backup start
5082 HMaster -D hbase.master.port=16003 -D hbase.master.info.port=16013 -D hbase.regionserver.port=16023 -D hbase.regi
onserver.info.port=16033 --backup start
4347 HMaster start
2431 HRegionServer -Dhbase.regionserver.port=16020 -Dhbase.regionserver.info.port=16030 start
aramber@localhost:~$ echo "status 'simple'" | hbase shell -n 2>/dev/null | grep -v Took | cut -c1-110
hbase:001:0> status 'simple'
active master:  localhost:16000 1789811249968
2 backup masters
    localhost:16002 1789811283786
    localhost:16003 1789811285924
1 live servers
    localhost:16020 1789811155600
        requestsPerSecond=0.0, numberOfOnlineRegions=2, usedHeapMB=54, maxHeapMB=512, numberOfStores=4, number
0 dead servers
Aggregate load: 0, regions: 2
hbase:002:0>
```

Figura 12. Arranque de los maestros de respaldo, `jps -m` (se ven los tres HMaster con sus puertos) y `status 'simple'`.

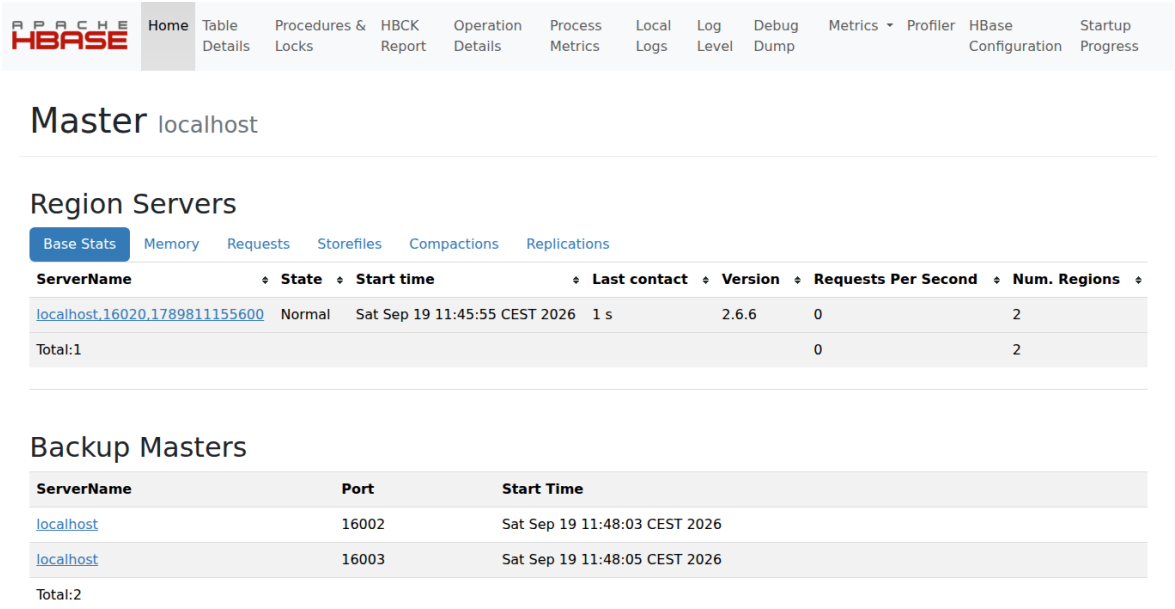


Figura 13. Interfaz web: sección *Backup Masters* con los dos maestros de respaldo.

A continuación se mata el **maestro principal** (el del puerto 16000). Los maestros de respaldo compiten por crear el nodo efímero `/hbase/master` en ZooKeeper y el que lo consigue

primero toma el control. En esta ejecución ganó el del **puerto 16003**, que pasa a ser el activo; el de 16002 sigue de respaldo.

```
aramber@localhost:~$ kill -9 4347    # HMaster principal (puerto 16000)
2177 HQuorumPeer start
4963 HMaster -D hbase.master.port=16002 -D hbase.master.info.port=16012 -D hbase.regionserver.port=16022 -D hb
5082 HMaster -D hbase.master.port=16003 -D hbase.master.info.port=16013 -D hbase.regionserver.port=16023 -D hb
2431 HRegionServer -Dhbase.regionserver.port=16020 -Dhbase.regionserver.info.port=16030 start
```

Figura 14. Muerte del maestro principal: quedan dos HMaster (16002 y 16003).

```
aramber@localhost:~$ echo "status 'simple'" | hbase shell -n 2>/dev/null | grep -v Took | cut -c1-110
hbase:001:0> status 'simple'
active master:  localhost:16003 1789811285924
1 backup masters
    localhost:16002 1789811283786
1 live servers
    localhost:16020 1789811155600
        requestsPerSecond=0.0, numberOfOnlineRegions=2, usedHeapMB=55, maxHeapMB=512, numberOfStores=4, number
0 dead servers
Aggregate load: 0, regions: 2
hbase:002:0>
```

Figura 15. `status 'simple'` tras la conmutación: maestro activo `localhost:16003` y 1 de respaldo (`16002`).

Como indica el enunciado, la interfaz web pasa a servirla el nuevo maestro activo, en su puerto de información (16013). En ella figura como respaldo el maestro 16002. En este entorno los maestros de respaldo no se cayeron solos tras arrancar, porque había memoria suficiente para las tres JVM de 512 MB.

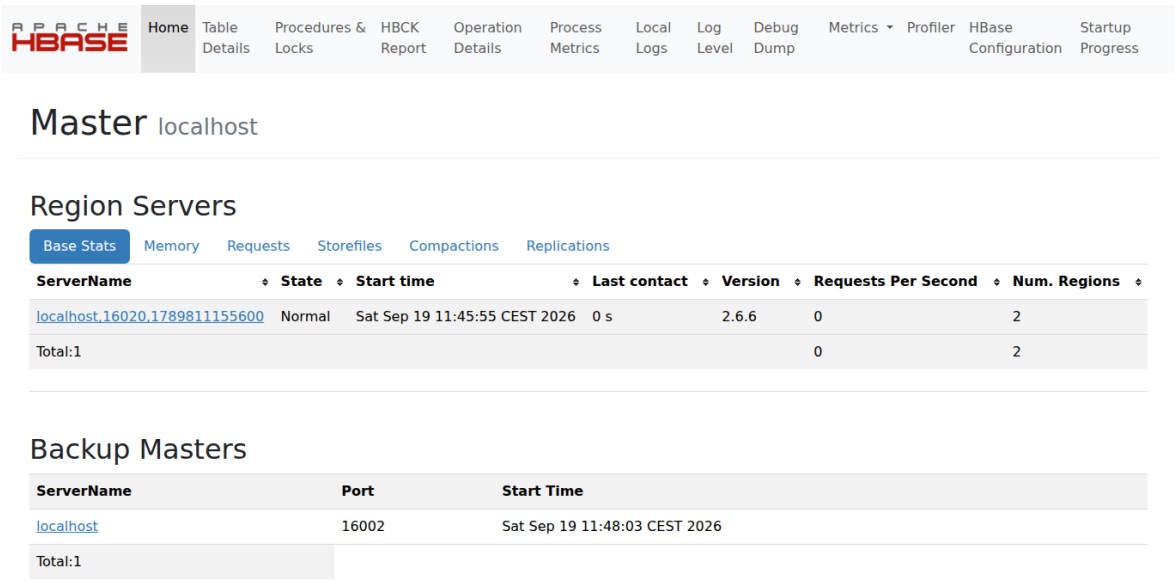


Figura 16. Interfaz web servida ahora por el maestro del puerto 16013.

A4. Ampliación a 5 servidores de regiones

Antes: un único servidor de regiones (`16020`).

```
aramber@localhost:~$ jps -m | grep HRegionServer | cut -c1-100
2431 HRegionServer -Dhbase.regionserver.port=16020 -Dhbase.regionserver.info.port=16030 start
aramber@localhost:~$ echo status | hbase shell -n 2>/dev/null | grep -v Took
hbase:001:0> status
1 active master, 1 backup masters, 1 servers, 0 dead, 2.0000 average load
hbase:002:0>
```

Figura 17. Estado previo: 1 servidor de regiones.

Con `local-regionservers.sh start 2 3 4 5` se lanzan cuatro servidores de regiones adicionales. Cada uno usa un *offset* de puertos (RPC 16022-16025 y web 16032-16035) para no chocar entre sí en la misma máquina. **Después**, `jps` muestra 5 `HRegionServer` y `status` informa de **5 servidores**. La carga media baja a 0,4 regiones por servidor: las 2 regiones de sistema se reparten ahora entre 5 servidores.

```
aramber@localhost:~$ local-regionservers.sh start 2 3 4 5
running regionserver, logging to /opt/hbase/bin/../logs/hbase-root-2-regionserver-vm.out
running regionserver, logging to /opt/hbase/bin/../logs/hbase-root-3-regionserver-vm.out
running regionserver, logging to /opt/hbase/bin/../logs/hbase-root-4-regionserver-vm.out
running regionserver, logging to /opt/hbase/bin/../logs/hbase-root-5-regionserver-vm.out
aramber@localhost:~$ sleep 30; jps -m | grep -v Jps | cut -c1-110
6704 HRegionServer -Dhbase.regionserver.port=16023 -Dhbase.regionserver.info.port=16033 start
2177 HQuorumPeer start
4963 HMaster -D hbase.master.port=16002 -D hbase.master.info.port=16012 -D hbase.regionserver.port=16022 -D hb
6825 HRegionServer -Dhbase.regionserver.port=16024 -Dhbase.regionserver.info.port=16034 start
5082 HMaster -D hbase.master.port=16003 -D hbase.master.info.port=16013 -D hbase.regionserver.port=16023 -D hb
6587 HRegionServer -Dhbase.regionserver.port=16022 -Dhbase.regionserver.info.port=16032 start
6942 HRegionServer -Dhbase.regionserver.port=16025 -Dhbase.regionserver.info.port=16035 start
2431 HRegionServer -Dhbase.regionserver.port=16020 -Dhbase.regionserver.info.port=16030 start
aramber@localhost:~$ echo status | hbase shell -n 2>/dev/null | grep -v Took
hbase:001:0> status
1 active master, 1 backup masters, 5 servers, 0 dead, 0.4000 average load
hbase:002:0>
```

Figura 18. Arranque de 4 servidores de regiones adicionales, `jps -m` y `status`.

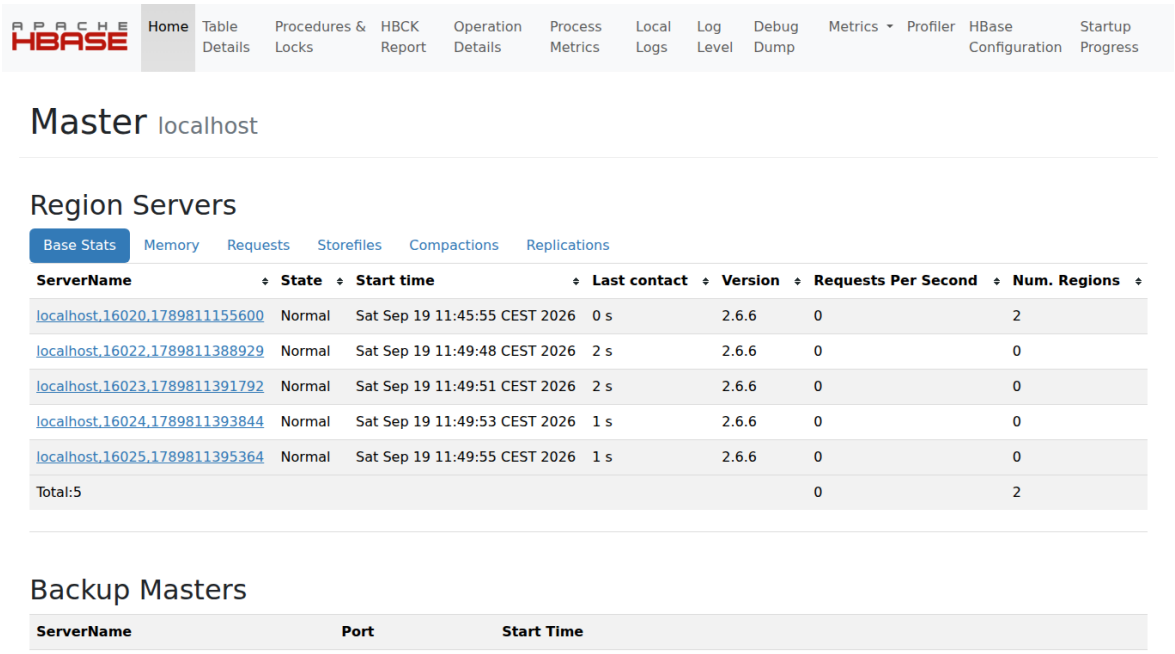


Figura 19. Interfaz web con los 5 servidores de regiones en estado *Normal*.

A5. Carga de datos, regiones y tabla hbase:meta

Se crea la tabla `clientes` con la familia de columnas `datos` y se insertan 10 filas. Las claves se eligen **lexicográficamente lejanas entre sí** y repartidas por el alfabeto (`a-001`, `c-002`, `e-003` … `x-010`), para que en la A6 la clave `k` las divida en dos mitades (5 y 5).

```
create 'clientes', 'datos'
put 'clientes', 'a-001', 'datos:nombre', 'Ana'
put 'clientes', 'c-002', 'datos:nombre', 'Carlos'
put 'clientes', 'e-003', 'datos:nombre', 'Elena'
put 'clientes', 'g-004', 'datos:nombre', 'Gonzalo'
put 'clientes', 'i-005', 'datos:nombre', 'Irene'
put 'clientes', 'k-006', 'datos:nombre', 'Kevin'
put 'clientes', 'm-007', 'datos:nombre', 'Marta'
```

```
put 'clientes', 'p-008', 'datos:nombre', 'Pablo'
put 'clientes', 's-009', 'datos:nombre', 'Sara'
put 'clientes', 'x-010', 'datos:nombre', 'Xavier'
count 'clientes'
scan 'clientes'
```

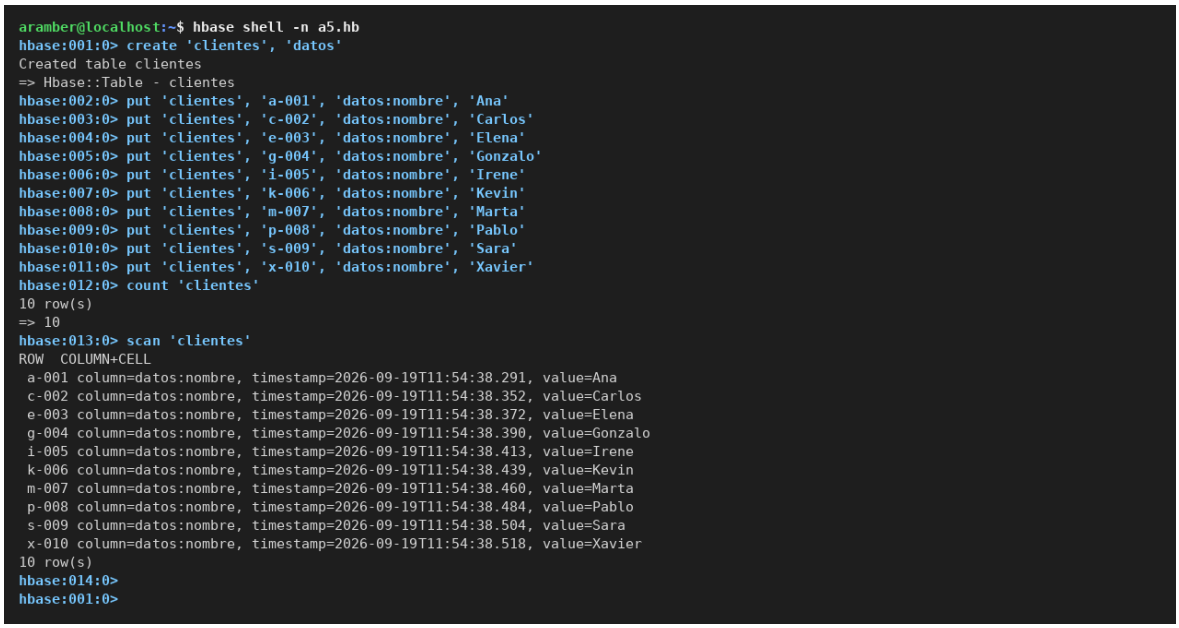


Figura 20. Creación de la tabla, inserción de 10 filas, `count` y `scan`.

¿En cuántas regiones se ha dividido la tabla? ¿Por qué?

En **una sola región**. Una tabla nueva sin *pre-splits* nace con una única región que abarca todo el espacio de claves (clave de inicio y de fin vacías). HBase sólo la divide automáticamente cuando crece por encima del umbral de la política de división (`hbase.hregion.max.filesize`, 10 GB por defecto, con la política `SteppingSplitPolicy`, que para las primeras regiones usa un umbral menor basado en el tamaño del *flush* de la memstore). Diez filas ocupan unos pocos KB, muy lejos de cualquier umbral.

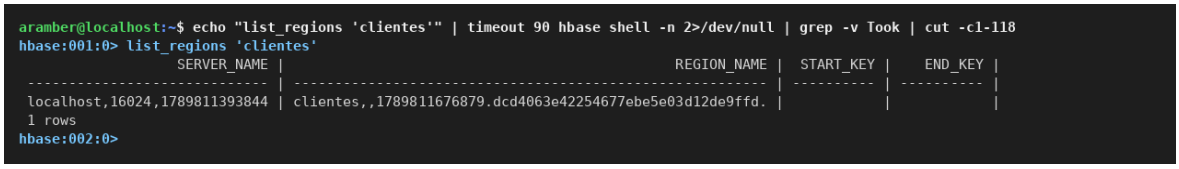


Figura 21. `list_regions 'clientes'`: una región, sin START_KEY ni END_KEY, alojada en `localhost:16024`.

¿Qué evidencias da la interfaz web?

La página de la tabla (*Table Details*) muestra en **Table Regions** una única región (`Name(1)`), con *Start Key* y *End Key* vacías, estado OPEN, el servidor que la aloja (`localhost:16034`, puerto web del servidor 16024) y las 10 escrituras. En **Regions by Region Server** se ve que ese servidor tiene 1 región de la tabla.

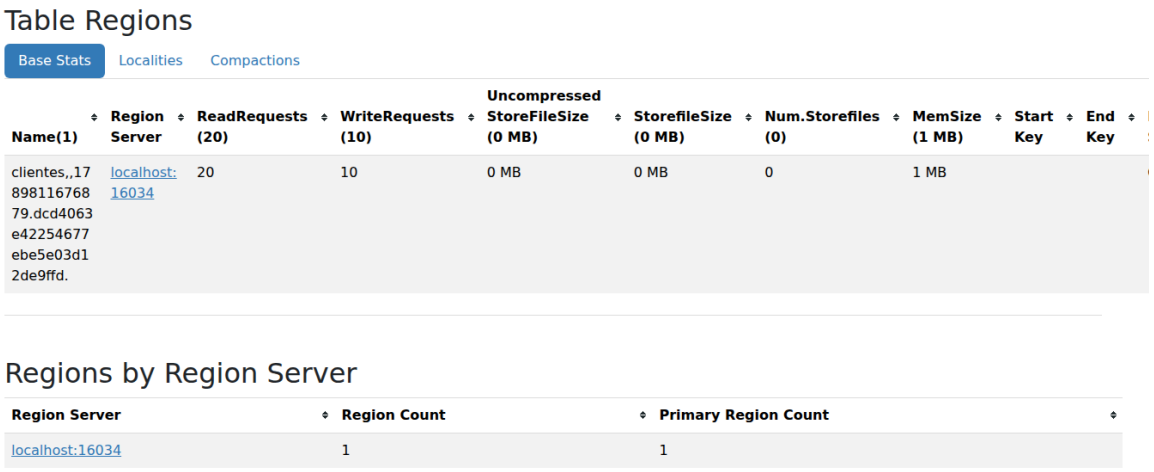


Figura 22. Interfaz web de la tabla `clientes`: una región alojada en un servidor.

Cometido de la tabla META e interpretación del scan

`hbase:meta` es el **catálogo de regiones** del clúster: guarda una fila por cada región de cada tabla y dice qué rango de claves cubre y qué servidor la sirve. Un cliente que quiere leer o escribir la fila `k-006` pregunta a ZooKeeper dónde está `hbase:meta`, busca en ella la región cuyo rango contiene la clave y va directamente al servidor que la aloja. El cliente guarda esa ubicación en caché. Por eso META es la pieza que responde a la pregunta anterior: el número de filas de región de una tabla en META es el número de regiones en que está dividida.

```
aramber@localhost:~$ echo "scan 'hbase:meta', {ROWPREFIXFILTER => 'clientes'}" | timeout 90 hbase shell -n 2>/dev/null | grep -v Took
hbase:001:0> scan 'hbase:meta', {ROWPREFIXFILTER => 'clientes'}
ROW  COLUMN+CELL
 clientes column=table:state, timestamp=2026-09-19T11:54:37.619, value=\x08\x00
 clientes,,1789811676879.dcd4063e42254677ebe5e03d12de9ffd. column=info:regioninfo, timestamp=2026-09-19T11:54:37.596,
value={ENCODED => dcd4063e42254677ebe5e03d12de9ffd, NAME => 'clientes,,1789811676879.dcd4063e42254677ebe5e03d12de9ffd.
', STARTKEY => '', ENDKEY => ''}
 clientes,,1789811676879.dcd4063e42254677ebe5e03d12de9ffd. column=info:seqnumDuringOpen, timestamp=2026-09-19T11:54:37
.596, value=\x00\x00\x00\x00\x00\x00\x00\x02
 clientes,,1789811676879.dcd4063e42254677ebe5e03d12de9ffd. column=info:server, timestamp=2026-09-19T11:54:37.596, valu
e=localhost:16024
 clientes,,1789811676879.dcd4063e42254677ebe5e03d12de9ffd. column=info:serverstartcode, timestamp=2026-09-19T11:54:37.
596, value=1789811393844
 clientes,,1789811676879.dcd4063e42254677ebe5e03d12de9ffd. column=info:sn, timestamp=2026-09-19T11:54:37.092, value=lo
calhost,16024,1789811393844
 clientes,,1789811676879.dcd4063e42254677ebe5e03d12de9ffd. column=info:state, timestamp=2026-09-19T11:54:37.596, value
=OPEN
2 row(s)
hbase:002:0>
```

Figura 23. `scan 'hbase:meta'` filtrado por la tabla `clientes`.

Interpretación de cada fila y columna del volcado:

Fila / columna	Valor	Significado
`clientes` · `table:state`	`\x08\x00`	Fila de estado de la tabla (no es una región). Es un protobuf serializado cuyo valor 0 equivale a ENABLED.
`clientes,,1789811676879.dcd40…` (clave de fila)	—	Nombre de la región: `<tabla>,<clave de inicio>,<timestamp de creación>.<hash codificado>.`. La clave de inicio vacía indica que es la primera región.
`info:regioninfo`	`{ENCODED => dcd40…, NAME => …, STARTKEY => '', ENDKEY => ''}`	Descriptor de la región: nombre codificado (MD5) y rango de claves [STARTKEY, ENDKEY). Ambos vacíos: cubre todo el espacio de claves.
`info:seqnumDuringOpen`	`\x00…\x02`	Número de secuencia del WAL en el

		momento de abrir la región. Se usa en la recuperación para saber desde qué punto reaplicar ediciones.
`info:server`	`localhost:16024`	Host y puerto del servidor de regiones que la sirve.
`info:serverstartcode`	`1789811393844`	Marca de tiempo de arranque de ese servidor. Distingue una instancia de otra en el mismo host:puerto tras un reinicio.
`info:sn`	`localhost,16024,1789811393844`	ServerName completo (host, puerto y *startcode*) asignado por el maestro.
`info:state`	`OPEN`	Estado de la asignación: la región está abierta y disponible.

Fuentes usadas para la interpretación: la *Apache HBase Reference Guide*, sección «Catalog Tables – hbase:meta» (https://hbase.apache.org/book.html#arch.catalog.meta), y el código de `org.apache.hadoop.hbase.HConstants`, donde se definen las columnas `info:regioninfo`, `info:server`, `info:serverstartcode`, `info:seqnumDuringOpen`, `info:sn` e `info:state`.

A6. Particionado forzado, movimiento de regiones, caída de un servidor y fusión

Split

Las claves van de `a-001` a `x-010`, así que se elige como punto de corte **`k`**. Deja 5 filas (a, c, e, g, i) a la izquierda y 5 (k, m, p, s, x) a la derecha. Estado *antes*: una región (véanse las figuras de la A5). Se ejecuta `split 'clientes', 'k'`:

```
aramber@localhost:~$ echo -e "split 'clientes', 'k'" | timeout 90 hbase shell -n 2>/dev/null | grep -v Took
hbase:001:0> split 'clientes', 'k'
hbase:002:0>
aramber@localhost:~$ sleep 10; echo "list_regions 'clientes'" | timeout 90 hbase shell -n 2>/dev/null | grep -v Took | cut -c1-118
hbase:001:0> list_regions 'clientes'
                  SERVER_NAME |                                                 REGION_NAME |  START_KEY |    END_KEY
 ---------------------------- | ---------------------------------------------------------- | ---------- | ----------
 localhost,16024,1789811393844 |  clientes,,1789811720587.6001b7b65764996f5aff2d4b554897ae. |            |          k
 localhost,16024,1789811393844 | clientes,k,1789811720587.38d4589cb9bb3355ba783cb5afd566a7. |          k |
 2 rows
hbase:002:0>
```

Figura 24. `split` y `list_regions` posterior: dos regiones, `[ , k)` y `[k, )`, ambas en `localhost:16024`.

En META (figura siguiente) aparecen ahora tres filas de región:

- **Región madre** `clientes,,1789811676879.dcd40…`: `info:state = CLOSED` y dos columnas nuevas, `info:splitA` e `info:splitB`, que apuntan a las hijas. Se mantiene hasta que el *CatalogJanitor* comprueba que las hijas ya no referencian sus ficheros y la elimina.
- **Hija A** `clientes,,1789811720587.6001b…`: rango `['', 'k')`, estado OPEN, servidor `localhost:16024`.
- **Hija B** `clientes,k,1789811720587.38d45…`: la clave de inicio `k` forma parte del nombre de la región. Rango `['k', '')`, OPEN, en `localhost:16024`.

```
aramber@localhost:~$ echo "scan 'hbase:meta', {ROWPREFIXFILTER => 'clientes', COLUMNS => ['info:regioninfo','info:server','info:state','i
nfo:splitA','info:splitB']}" | timeout 90 hbase shell -n 2>/dev/null | grep -v Took
hbase:001:0> scan 'hbase:meta', {ROWPREFIXFILTER => 'clientes', COLUMNS => ['info:regioninfo','info:server','info:stat
e','info:splitA','info:splitB']}
ROW  COLUMN+CELL
 clientes,,1789811676879.dcd4063e42254677ebe5e03d12de9ffd. column=info:regioninfo, timestamp=2026-09-19T11:55:21.053,
value={ENCODED => dcd4063e42254677ebe5e03d12de9ffd, NAME => 'clientes,,1789811676879.dcd4063e42254677ebe5e03d12de9ffd.
', STARTKEY => '', ENDKEY => '', OFFLINE => true, SPLIT => true}
 clientes,,1789811676879.dcd4063e42254677ebe5e03d12de9ffd. column=info:server, timestamp=2026-09-19T11:54:37.596, valu
e=localhost:16024
 clientes,,1789811676879.dcd4063e42254677ebe5e03d12de9ffd. column=info:splitA, timestamp=2026-09-19T11:55:21.053, valu
e={ENCODED => 6001b7b65764996f5aff2d4b554897ae, NAME => 'clientes,,1789811720587.6001b7b65764996f5aff2d4b554897ae.', S
TARTKEY => '', ENDKEY => 'k'}
 clientes,,1789811676879.dcd4063e42254677ebe5e03d12de9ffd. column=info:splitB, timestamp=2026-09-19T11:55:21.053, valu
e={ENCODED => 38d4589cb9bb3355ba783cb5afd566a7, NAME => 'clientes,k,1789811720587.38d4589cb9bb3355ba783cb5afd566a7.',
STARTKEY => 'k', ENDKEY => ''}
 clientes,,1789811676879.dcd4063e42254677ebe5e03d12de9ffd. column=info:state, timestamp=2026-09-19T11:55:20.941, value
=CLOSED
 clientes,,1789811720587.6001b7b65764996f5aff2d4b554897ae. column=info:regioninfo, timestamp=2026-09-19T11:55:21.627,
value={ENCODED => 6001b7b65764996f5aff2d4b554897ae, NAME => 'clientes,,1789811720587.6001b7b65764996f5aff2d4b554897ae.
', STARTKEY => '', ENDKEY => 'k'}
 clientes,,1789811720587.6001b7b65764996f5aff2d4b554897ae. column=info:server, timestamp=2026-09-19T11:55:21.627, valu
e=localhost:16024
 clientes,,1789811720587.6001b7b65764996f5aff2d4b554897ae. column=info:state, timestamp=2026-09-19T11:55:21.627, value
=OPEN
 clientes,k,1789811720587.38d4589cb9bb3355ba783cb5afd566a7. column=info:regioninfo, timestamp=2026-09-19T11:55:21.625,
 value={ENCODED => 38d4589cb9bb3355ba783cb5afd566a7, NAME => 'clientes,k,1789811720587.38d4589cb9bb3355ba783cb5afd566a
7.', STARTKEY => 'k', ENDKEY => ''}
 clientes,k,1789811720587.38d4589cb9bb3355ba783cb5afd566a7. column=info:server, timestamp=2026-09-19T11:55:21.625, val
ue=localhost:16024
 clientes,k,1789811720587.38d4589cb9bb3355ba783cb5afd566a7. column=info:state, timestamp=2026-09-19T11:55:21.625, valu
e=OPEN
3 row(s)
hbase:002:0>
```

Figura 25. `hbase:meta` tras el split: región madre cerrada con `splitA`/`splitB` y las dos hijas abiertas.

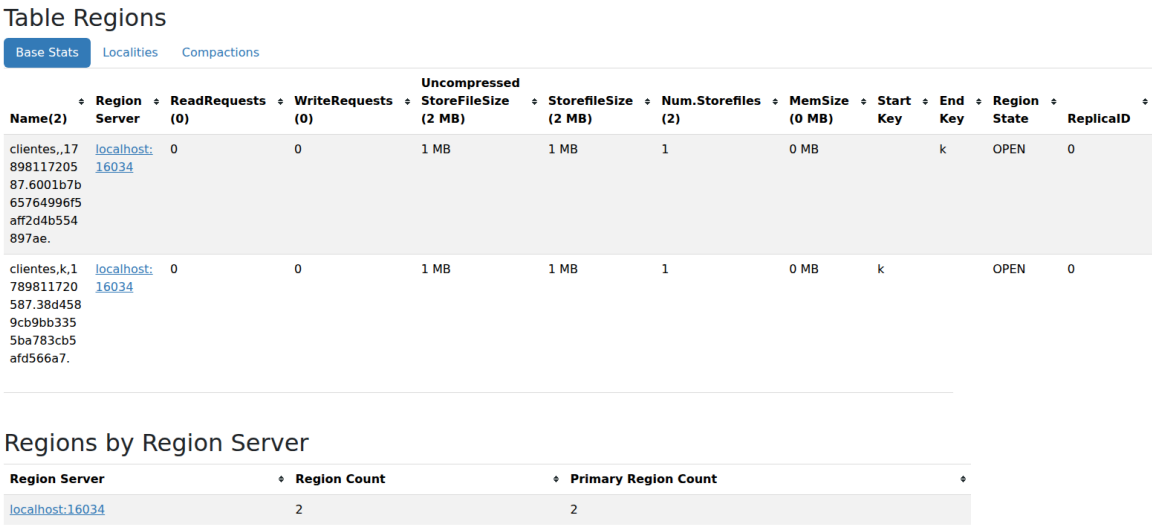


Figura 26. Interfaz web: la tabla tiene 2 regiones (End Key `k` / Start Key `k`), ambas en el servidor 16024 (web 16034).

Tras el split, las dos regiones siguen en el mismo servidor (`16024`): dividir una región no la mueve. Moverla lo decide después el balanceador o el administrador.

Move

Se mueve la hija B (nombre codificado `38d4589cb9bb3355ba783cb5afd566a7`) al servidor `localhost,16022,1789811388929`, obtenido de `status 'simple'`/interfaz web:

```
aramber@localhost:~$ echo "move '38d4589cb9bb3355ba783cb5afd566a7', 'localhost,16022,1789811388929'" | timeout 90 hbase shell -n 2>/dev/n
ull | grep -v Took
hbase:001:0> move '38d4589cb9bb3355ba783cb5afd566a7', 'localhost,16022,1789811388929'
hbase:002:0>
aramber@localhost:~$ sleep 8; echo "list_regions 'clientes'" | timeout 90 hbase shell -n 2>/dev/null | grep -v Took | cut -c1-118
hbase:001:0> list_regions 'clientes'
                  SERVER_NAME |                                                  REGION_NAME |  START_KEY |    END_KEY
 -------------------------- | ---------------------------------------------------------- | ---------- | ----------
 localhost,16024,1789811393844 |  clientes,,1789811720587.6001b7b65764996f5aff2d4b554897ae. |            |          k
 localhost,16022,1789811388929 | clientes,k,1789811720587.38d4589cb9bb3355ba783cb5afd566a7. |          k |
 2 rows
hbase:002:0>
```

Figura 27. `move` de la región `[k, )` al servidor 16022: ahora cada mitad está en un servidor distinto.

Caída de un servidor de regiones

Se elige el servidor **16022**, que aloja la región `[k, )`. Antes de matarlo hay 5 servidores vivos:

```
aramber@localhost:~$ jps -m | grep 'HRegionServer' | cut -c1-100
6704 HRegionServer -Dhbase.regionserver.port=16023 -Dhbase.regionserver.info.port=16033 start
6825 HRegionServer -Dhbase.regionserver.port=16024 -Dhbase.regionserver.info.port=16034 start
6587 HRegionServer -Dhbase.regionserver.port=16022 -Dhbase.regionserver.info.port=16032 start
6942 HRegionServer -Dhbase.regionserver.port=16025 -Dhbase.regionserver.info.port=16035 start
2431 HRegionServer -Dhbase.regionserver.port=16020 -Dhbase.regionserver.info.port=16030 start
```

Figura 28. Los 5 HRegionServer antes de la caída.

```
aramber@localhost:~$ kill -9 6587    # HRegionServer del puerto 16022 (aloja la región clientes,k)
6704 HRegionServer -Dhbase.regionserver.port=16023 -Dhbase.regionserver.info.port=16033 start
6825 HRegionServer -Dhbase.regionserver.port=16024 -Dhbase.regionserver.info.port=16034 start
6942 HRegionServer -Dhbase.regionserver.port=16025 -Dhbase.regionserver.info.port=16035 start
2431 HRegionServer -Dhbase.regionserver.port=16020 -Dhbase.regionserver.info.port=16030 start
```

Figura 29. `kill -9` del HRegionServer del puerto 16022: quedan 4.

El maestro detecta la caída cuando expira la sesión de ZooKeeper del servidor. Lanza entonces un *ServerCrashProcedure*: divide el WAL del servidor muerto, reasigna sus regiones a servidores vivos y estos reaplican las ediciones pendientes. Pasados unos segundos, `status` informa de **4 servidores y 1 caído**, y `list_regions` muestra que la región `[k, )` **ha pasado de 16022 a 16023**. Un `scan` desde `k` devuelve las 5 filas (Kevin, Marta, Pablo, Sara, Xavier): **no se ha perdido ningún dato**, porque los ficheros de la región están en el almacenamiento compartido (aquí el sistema de ficheros local; en producción, HDFS) y no en el proceso que murió.

```
aramber@localhost:~$ echo status | timeout 90 hbase shell -n 2>/dev/null | grep -v Took
hbase:001:0> status
1 active master, 1 backup masters, 4 servers, 1 dead, 1.0000 average load
hbase:002:0>
aramber@localhost:~$ echo "list_regions 'clientes'" | timeout 90 hbase shell -n 2>/dev/null | grep -v Took | cut -c1-118
hbase:001:0> list_regions 'clientes'
                  SERVER_NAME |                                                  REGION_NAME |  START_KEY |    END_KEY
 -------------------------- | ---------------------------------------------------------- | ---------- | ----------
 localhost,16024,1789811393844 |  clientes,,1789811720587.6001b7b65764996f5aff2d4b554897ae. |            |          k
 localhost,16023,1789811391792 | clientes,k,1789811720587.38d4589cb9bb3355ba783cb5afd566a7. |          k |
 2 rows
hbase:002:0>
aramber@localhost:~$ echo "scan 'clientes', {STARTROW => 'k'}" | timeout 90 hbase shell -n 2>/dev/null | grep -v Took
hbase:001:0> scan 'clientes', {STARTROW => 'k'}
ROW  COLUMN+CELL
 k-006 column=datos:nombre, timestamp=2026-09-19T11:54:38.439, value=Kevin
 m-007 column=datos:nombre, timestamp=2026-09-19T11:54:38.460, value=Marta
 p-008 column=datos:nombre, timestamp=2026-09-19T11:54:38.484, value=Pablo
 s-009 column=datos:nombre, timestamp=2026-09-19T11:54:38.504, value=Sara
 x-010 column=datos:nombre, timestamp=2026-09-19T11:54:38.518, value=Xavier
5 row(s)
hbase:002:0>
```

Figura 30. Después de la caída: nueva ubicación de la región y comprobación de los datos con `scan`.

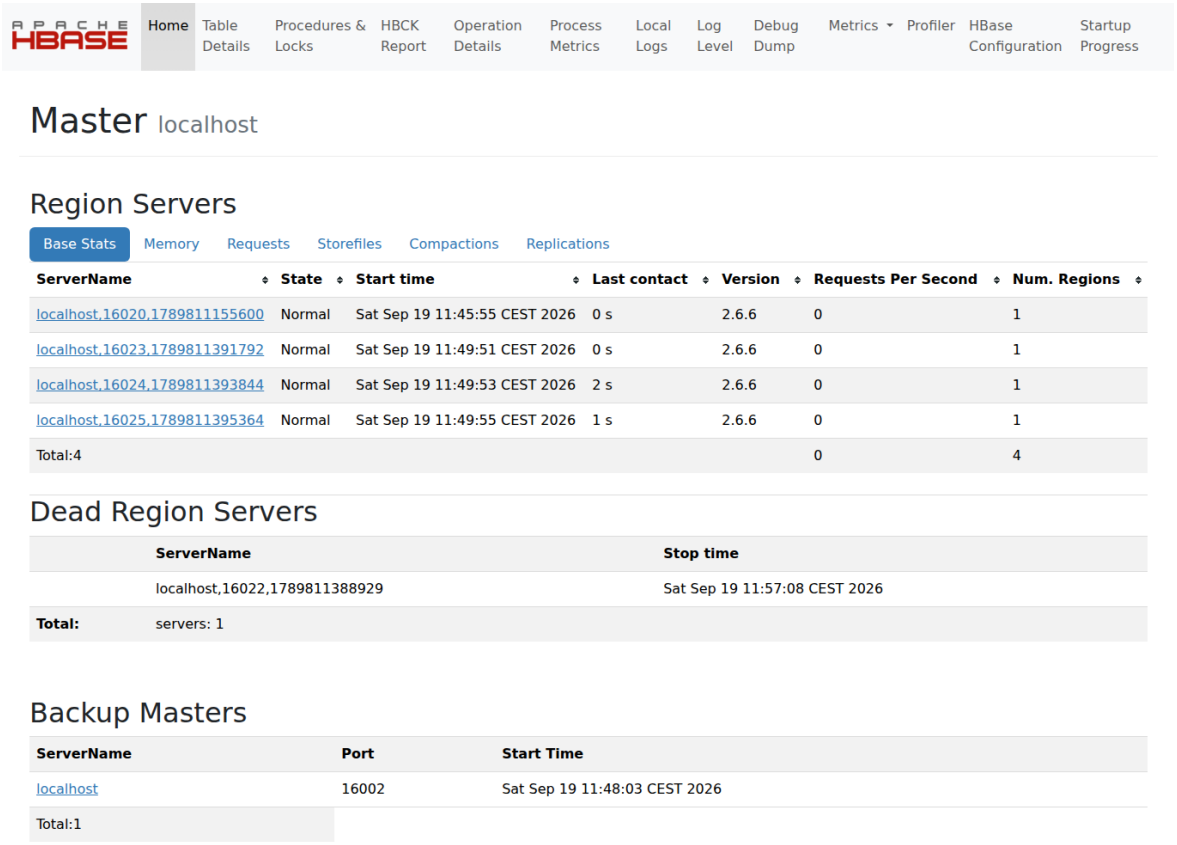


Figura 31. Interfaz web: 4 servidores activos y `localhost,16022,…` en *Dead Region Servers*.

Merge

Por último, se fusionan las dos hijas con `merge_region` a partir de sus nombres codificados. Resultado: la tabla vuelve a tener **una sola región** (`clientes,,1789811720588.90bc2…`, sin claves de inicio ni fin), **alojada en `localhost:16024`**, y `count` sigue devolviendo 10 filas.

```
aramber@localhost:~$ echo "merge_region '6001b7b65764996f5aff2d4b554897ae', '38d4589cb9bb3355ba783cb5afd566a7'" | timeout 90 hbase shell
-n 2>/dev/null | grep -v Took
hbase:001:0> merge_region '6001b7b65764996f5aff2d4b554897ae', '38d4589cb9bb3355ba783cb5afd566a7'
hbase:002:0>
aramber@localhost:~$ sleep 10; echo "list_regions 'clientes'" | timeout 90 hbase shell -n 2>/dev/null | grep -v Took | cut -c1-118
hbase:001:0> list_regions 'clientes'
                  SERVER_NAME |                                                 REGION_NAME |  START_KEY |    END_KEY |
 --------------------------- | ----------------------------------------------------------- | ---------- | ---------- |
 localhost,16024,1789811393844 | clientes,,1789811720588.90bc2a42d2ae73fa1006e42e06a1b507. |            |            |
 1 rows
hbase:002:0>
aramber@localhost:~$ echo "count 'clientes'" | timeout 90 hbase shell -n 2>/dev/null | grep -v Took
hbase:001:0> count 'clientes'
10 row(s)
=> 10
hbase:002:0>
```

Figura 32. `merge_region`, `list_regions` (1 región) y `count` (10 filas).

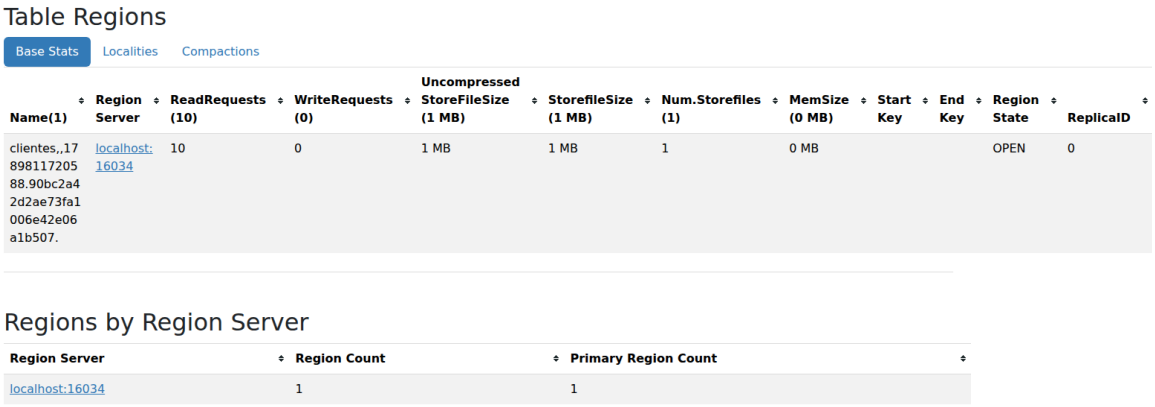


Figura 33. Interfaz web tras la fusión: una región en el servidor 16024 (web 16034).

A7 (opcional). Instalación completamente distribuida

Planteamiento. Como no se dispone de varias máquinas, cada nodo del clúster se simula con un **contenedor Docker** (Docker Desktop en el portátil, backend WSL2), tal como permite el enunciado. Los contenedores comparten una red de Docker y se resuelven por nombre (`namenode`, `zk1`, `hmaster`…), como lo harían hosts reales. Siguiendo la guía de referencia de HBase para el modo completamente distribuido, el clúster tiene **siete nodos**:

Nodo (contenedor)	Imagen	Papel
`namenode`	`apache/hadoop:3.3.6`	HDFS NameNode: metadatos del sistema de ficheros (web en el puerto 9870)
`datanode1`, `datanode2`	`apache/hadoop:3.3.6`	HDFS DataNodes: guardan los bloques, replicados en ambos (factor 2)
`zk1`	`zookeeper:3.8`	ZooKeeper externo: coordinación, registro de RegionServers y ubicación de `hbase:meta`
`hmaster`	`mbda/hbase:2.5.10` (propia)	HBase Master: asigna regiones y gestiona el esquema (web en el puerto 16010)
`rs1`, `rs2`	`mbda/hbase:2.5.10` (propia)	RegionServers: sirven las regiones de las tablas

La diferencia con el modo pseudodistribuido del resto de la actividad es que **cada proceso corre en un nodo distinto** y los datos **no están en el disco local sino en HDFS** (`hbase.rootdir=hdfs://namenode:8020/hbase`), replicados entre dos DataNodes. ZooKeeper es un servicio independiente (`HBASE_MANAGES_ZK=false`), no el que HBase arranca por su cuenta.

Recursos. Por la limitación de memoria del portátil se ha usado un solo nodo de ZooKeeper (en producción serían 3 o 5 para tener *quorum*) y *heaps* reducidos: 256 MB para Hadoop y 384 MB por proceso de HBase. El clúster completo ocupa unos **1,7 GB de RAM**, lo que cabe en el límite de Docker Desktop (3,6 GB).

Configuración

Imagen de HBase (HBase 2.5.10 compilado para Hadoop 3, sobre Java 11):

```
FROM eclipse-temurin:11-jre
```

```
ARG HBASE_VERSION=2.5.10
RUN apt-get update && apt-get install -y --no-install-recommends curl procps && rm -rf
/var/lib/apt/lists/* \
 && curl -fsSL https://archive.apache.org/dist/hbase/${HBASE_VERSION}/hbase-$
{HBASE_VERSION}-hadoop3-bin.tar.gz | tar -xz -C /opt \
 && ln -s /opt/hbase-${HBASE_VERSION}-hadoop3 /opt/hbase
ENV HBASE_HOME=/opt/hbase PATH=$PATH:/opt/hbase/bin HBASE_MANAGES_ZK=false
HBASE_HEAPSIZE=512m
WORKDIR /opt/hbase
```

`hbase-site.xml`, montado en los tres nodos de HBase:

```
<?xml version="1.0"?>
<configuration>
  <!-- Modo completamente distribuido: master y regionservers en contenedores distintos -->
  <property><name>hbase.cluster.distributed</name><value>true</value></property>
  <!-- Datos en HDFS (NameNode del clúster), no en el sistema de ficheros local -->
  <property><name>hbase.rootdir</name><value>hdfs://namenode:8020/hbase</value></property>
  <!-- ZooKeeper externo (contenedor zk1), no gestionado por HBase -->
  <property><name>hbase.zookeeper.quorum</name><value>zk1</value></property>
  <property><name>hbase.zookeeper.property.clientPort</name><value>2181</value></property>
  <!-- Factor de replicación de los ficheros que escribe HBase = nº de DataNodes -->
  <property><name>dfs.replication</name><value>2</value></property>
  <property><name>dfs.client.use.datanode.hostname</name><value>true</value></property>
  <property><name>hbase.wal.provider</name><value>filesystem</value></property>
</configuration>
```

`docker-compose.yml` con los siete nodos y **`hadoop.env`** (configuración de HDFS en formato de la imagen oficial de Apache Hadoop):

```
name: hbase-distribuido
# Clúster HBase completamente distribuido simulado con contenedores (un contenedor = un nodo):
#   HDFS: namenode + datanode1 + datanode2  |  ZooKeeper: zk1  |  HBase: hmaster + rs1 + rs2
x-hadoop: &hadoop
  image: apache/hadoop:3.3.6
  env_file: ./hadoop.env
  environment: &hadoopenv
    HADOOP_HEAPSIZE_MAX: 256m
x-hbase: &hbase
  build: ./hbase
  image: mbda/hbase:2.5.10
  restart: on-failure
  environment:
    HBASE_HEAPSIZE: 384m
  volumes:
    - ./hbase/hbase-site.xml:/opt/hbase/conf/hbase-site.xml:ro
    - ./work:/work
  depends_on: [namenode, datanode1, datanode2, zk1]
services:
  namenode:
    <<: *hadoop
    hostname: namenode
    command: ["hdfs", "namenode"]
    environment:
      <<: *hadoopenv
      ENSURE_NAMENODE_DIR: /tmp/hadoop-hadoop/dfs/name
    ports: ["9870:9870"]
  datanode1:
    <<: *hadoop
    hostname: datanode1
    command: ["hdfs", "datanode"]
  datanode2:
    <<: *hadoop
    hostname: datanode2
    command: ["hdfs", "datanode"]
```

```
  zk1:
    image: zookeeper:3.8
    hostname: zk1
  hmaster:
    <<: *hbase
    hostname: hmaster
    command: ["hbase", "master", "start"]
    ports: ["16010:16010"]
  rs1:
    <<: *hbase
    hostname: rs1
    command: ["hbase", "regionserver", "start"]
    ports: ["16030:16030"]
  rs2:
    <<: *hbase
    hostname: rs2
    command: ["hbase", "regionserver", "start"]
    ports: ["16031:16030"]
```

```
CORE-SITE.XML_fs.default.name=hdfs://namenode
CORE-SITE.XML_fs.defaultFS=hdfs://namenode
HDFS-SITE.XML_dfs.namenode.rpc-address=namenode:8020
HDFS-SITE.XML_dfs.replication=2
HDFS-SITE.XML_dfs.permissions.enabled=false
HDFS-SITE.XML_dfs.datanode.use.datanode.hostname=true
HDFS-SITE.XML_dfs.client.use.datanode.hostname=true
```

Despliegue y verificación

Desde el terminal integrado de Docker Desktop (PowerShell), en la carpeta `A7_hbase_distribuido`: `docker compose build` construye la imagen de HBase y `docker compose up -d` levanta el clúster. Las salidas que siguen son las reales, redirigidas a ficheros de texto en `work/` y reproducidas tal cual (en algunas se muestra un extracto).

```
aramber@localhost:~$ docker compose build   # imagen mbda/hbase:2.5.10 (Dockerfile propio)
aramber@localhost:~$ docker compose up -d
aramber@localhost:~$ docker compose ps
NAME                          IMAGE                 STATUS         PORTS
hbase-distribuido-datanode1-1   apache/hadoop:3.3.6   Up 4 minutes
hbase-distribuido-datanode2-1   apache/hadoop:3.3.6   Up 5 minutes
hbase-distribuido-hmaster-1     mbda/hbase:2.5.10     Up 4 minutes   0.0.0.0:16010->16010/tcp, [::]:16010->16010/tcp
hbase-distribuido-namenode-1    apache/hadoop:3.3.6   Up 5 minutes   0.0.0.0:9870->9870/tcp, [::]:9870->9870/tcp
hbase-distribuido-rs1-1         mbda/hbase:2.5.10     Up 4 minutes   0.0.0.0:16030->16030/tcp, [::]:16030->16030/tcp
hbase-distribuido-rs2-1         mbda/hbase:2.5.10     Up 4 minutes   0.0.0.0:16031->16030/tcp, [::]:16031->16030/tcp
hbase-distribuido-zk1-1         zookeeper:3.8         Up 5 minutes   2181/tcp, 2888/tcp, 3888/tcp, 8080/tcp
```

Figura 34. Los siete nodos en marcha. Sólo se publican al anfitrión los puertos web (9870, 16010, 16030/16031) y el resto de la comunicación va por la red interna.

```
aramber@localhost:~$ docker compose exec namenode hdfs dfsadmin -report   (extracto)
Configured Capacity: 2162202353664 (1.97 TB)
Present Capacity: 2034821967888 (1.85 TB)
DFS Used: 69518 (67.89 KB)
Live datanodes (2):
Name: 172.18.0.3:9866 (hbase-distribuido-datanode2-1.hbase-distribuido_default)
Hostname: datanode2
Configured Capacity: 1081101176832 (1006.85 GB)
DFS Used: 34759 (33.94 KB)
Last contact: Sun Sep 20 08:20:32 UTC 2026
Num of Blocks: 29
Name: 172.18.0.5:9866 (hbase-distribuido-datanode1-1.hbase-distribuido_default)
Hostname: datanode1
Configured Capacity: 1081101176832 (1006.85 GB)
DFS Used: 34759 (33.94 KB)
Last contact: Sun Sep 20 08:20:32 UTC 2026
Num of Blocks: 29
```

Figura 35. HDFS funcional: el NameNode ve 2 DataNodes vivos (datanode1 y datanode2).

```
aramber@localhost:~$ docker compose exec hmaster hbase shell -n /work/estado.hb   # status 'simple'
hbase:001:0> status 'simple'
active master:  hmaster:16000 1789892494450
0 backup masters
2 live servers
    hbase-distribuido-rs1-1.hbase-distribuido_default:16020 1789892494450
        requestsPerSecond=0.0, numberOfOnlineRegions=0, usedHeapMB=84, maxHeapMB=384, numberOfStores=0, numberOfStoref
iles=0, storefileUncom …
    hbase-distribuido-rs2-1.hbase-distribuido_default:16020 1789892494480
        requestsPerSecond=0.0, numberOfOnlineRegions=2, usedHeapMB=63, maxHeapMB=384, numberOfStores=4, numberOfStoref
iles=3, storefileUncom …
0 dead servers
Aggregate load: 0, regions: 2
Took 1.7057 seconds
hbase:002:0> exit
```

Figura 36. Estado de HBase: 1 master activo (`hmaster`), 2 RegionServers vivos (rs1 y rs2) y 0 caídos.

Prueba con una tabla pre-particionada. Se crea `clientes_dist` con dos puntos de corte (`g` y `n`), es decir, tres regiones, y se insertan claves que caen en cada una. Así se comprueba que el Master **reparte las regiones entre los dos RegionServers**:

```
create 'clientes_dist', 'datos', SPLITS => ['g', 'n']
put 'clientes_dist', 'a-001', 'datos:nombre', 'Ana'
put 'clientes_dist', 'h-002', 'datos:nombre', 'Hugo'
put 'clientes_dist', 'p-003', 'datos:nombre', 'Paula'
put 'clientes_dist', 'z-004', 'datos:nombre', 'Zoe'
scan 'clientes_dist'
list_regions 'clientes_dist'
flush 'clientes_dist'
exit
```

```
aramber@localhost:~$ docker compose exec hmaster hbase shell -n /work/prueba.hb
hbase:001:0> create 'clientes_dist', 'datos', SPLITS => ['g', 'n']
Created table clientes_dist
Took 11.2027 seconds
=> Hbase::Table - clientes_dist
hbase:002:0> put 'clientes_dist', 'a-001', 'datos:nombre', 'Ana'
Took 0.5114 seconds
hbase:003:0> put 'clientes_dist', 'h-002', 'datos:nombre', 'Hugo'
Took 0.4427 seconds
hbase:004:0> put 'clientes_dist', 'p-003', 'datos:nombre', 'Paula'
Took 0.0860 seconds
hbase:005:0> put 'clientes_dist', 'z-004', 'datos:nombre', 'Zoe'
Took 0.0170 seconds
hbase:006:0> scan 'clientes_dist'
ROW  COLUMN+CELL
 a-001 column=datos:nombre, timestamp=2026-09-20T08:24:45.765, value=Ana
 h-002 column=datos:nombre, timestamp=2026-09-20T08:24:46.232, value=Hugo
 p-003 column=datos:nombre, timestamp=2026-09-20T08:24:46.417, value=Paula
 z-004 column=datos:nombre, timestamp=2026-09-20T08:24:46.480, value=Zoe
4 row(s)
Took 0.5586 seconds
hbase:007:0> list_regions 'clientes_dist'
 SERVER_NAME | REGION_NAME | START_KEY | END_KEY | SIZE | REQ | LOCALITY |
 ------ | ------ | ------ | ------ | ----- | ----- | ------ |
 rs2-1,16020,1789892494480 | clientes_dist,,1789892676409.067c0725753189336359ed30920f188b. | | g | 0 | 0 | 0.0 |
 rs1-1,16020,1789892494450 | clientes_dist,g,1789892676409.7f6d9f8c514378c54bf741dc09b90686. | g | n | 0 | 0 | 0.0 |
 rs1-1,16020,1789892494450 | clientes_dist,n,1789892676409.ae3ba522d808d68565a92b3eda06a519. | n | | 0 | 0 | 0.0 |
 3 rows
Took 0.3331 seconds
hbase:008:0> flush 'clientes_dist'
Took 2.1962 seconds
hbase:009:0> exit
```

Figura 37. Creación, inserción y lectura; `list_regions` muestra la región [−, g) en rs2 y las regiones [g, n) y [n, +) en rs1.

```
aramber@localhost:~$ docker compose exec namenode hdfs dfs -ls -R /hbase/data/default   (ficheros de datos)
-rw-r--r--   2 root supergroup       4909 2026-09-20 08:24 /hbase/data/default/clientes_dist/067c0725753189336359ed309
20f188b/datos/20778dfd4d8b48e8b5f3c70a07903648
-rw-r--r--   2 root supergroup       4910 2026-09-20 08:24 /hbase/data/default/clientes_dist/7f6d9f8c514378c54bf741dc0
9b90686/datos/94c734414640479f81b4aa4f27ca53bd
-rw-r--r--   2 root supergroup       4953 2026-09-20 08:24 /hbase/data/default/clientes_dist/ae3ba522d808d68565a92b3ed
a06a519/datos/63e96323095d49f296b6e975de9fae40

aramber@localhost:~$ docker compose exec namenode hdfs fsck /hbase/data/default -files -blocks -locations   (extracto)
FSCK started by hadoop (auth:SIMPLE) from /172.18.0.4 for path /hbase/data/default at Sun Sep 20 08:25:01 UTC 2026
/hbase/data/default/clientes_dist/067c0725753189336359ed30920f188b/datos/20778dfd4d8b48e8b5f3c70a07903648 4909 bytes,
replicated: replication=2, 1 blo …
0. BP-438018575-172.18.0.4-1789892143686:blk_1073741881_1059 len=4909 Live_repl=2  [172.18.0.5:9866, 172.18.0.3:9866]
/hbase/data/default/clientes_dist/7f6d9f8c514378c54bf741dc09b90686/datos/94c734414640479f81b4aa4f27ca53bd 4910 bytes,
replicated: replication=2, 1 blo …
0. BP-438018575-172.18.0.4-1789892143686:blk_1073741882_1060 len=4910 Live_repl=2  [172.18.0.5:9866, 172.18.0.3:9866]
/hbase/data/default/clientes_dist/ae3ba522d808d68565a92b3eda06a519/datos/63e96323095d49f296b6e975de9fae40 4953 bytes,
replicated: replication=2, 1 blo …
0. BP-438018575-172.18.0.4-1789892143686:blk_1073741883_1061 len=4953 Live_repl=2  [172.18.0.5:9866, 172.18.0.3:9866]
Status: HEALTHY
```

Figura 38. Tras `flush`, cada región tiene su HFile en HDFS, y `fsck` confirma que cada bloque tiene 2 réplicas vivas, una en cada DataNode. Estado HEALTHY.

```
aramber@localhost:~$ docker compose exec zk1 zkCli.sh -server localhost:2181 ls /hbase/rs
Connecting to localhost:2181
WatchedEvent state:SyncConnected type:None path:null
[hbase-distribuido-rs1-1.hbase-distribuido_default,16020,1789892494450, hbase-distribuido-rs2-1.hbase-distribuido_defa
ult,16020,1789892494480]
```

Figura 39. ZooKeeper registra los dos RegionServers en el znode `/hbase/rs`: es el mecanismo con el que el Master detecta altas y caídas.

Incidencia y ajuste. En el primer arranque `dfsadmin -report` indicaba 29 bloques *under-replicated*. HBase escribe en HDFS con su propia configuración de cliente, cuyo factor de replicación por defecto es 3, y sólo hay 2 DataNodes. Se añadió `dfs.replication=2` a `hbase-site.xml`, se reiniciaron los nodos de HBase y se ajustaron los ficheros existentes con `hdfs dfs -setrep -R 2 /hbase`. Después, `fsck` devuelve `replication=2` y `HEALTHY`. Al reiniciar, el Master ejecutó el *ServerCrashProcedure* de los RegionServers anteriores y **reprodujo sus WAL desde HDFS**. Es precisamente la ventaja del modo distribuido: si cae un RegionServer, otro puede recuperar sus regiones porque los datos y el registro de escrituras no están en su disco local.

Parada. `docker compose down` detiene y elimina los contenedores. Con `-v` también se borran los datos de HDFS. Los ficheros de configuración quedan en la carpeta `A7_hbase_distribuido` de la actividad para reproducir el clúster.

Referencias

[1] Apache HBase Reference Guide – Getting Started, Pseudo-Distributed Local Install y Fully Distributed. https://hbase.apache.org/book.html#quickstart_pseudo

[2] Apache HBase Reference Guide – Catalog Tables (hbase:meta). https://hbase.apache.org/book.html#arch.catalog.meta

[3] Apache HBase Reference Guide – Region Splitting y Region Merge. https://hbase.apache.org/book.html#arch.region.splits

[4] HBase shell commands (split, move, merge_region, list_regions). https://learnhbase.wordpress.com/2013/03/02/hbase-shell-commands/